

h8_rotcluster.py

H8 Rotational Symmetry: Super Cluster Assembly and Level-Coloured Overlay

MyHatTileStuff/h8_rotcluster.py — Technical Reference

June 17, 2026

Purpose

`h8_rotcluster.py` bundles the H8 rotational-symmetry findings into a single self-contained script (NumPy only). For arrangement #13 — the gap-free centre of type $v[12]$ at world coordinate $C_{13} = (-68.5, 0.866\dots)$ — it performs four tasks in sequence:

1. **Super cluster construction.** Loads the base H8 super tile from `from_python/SUPER_SUPER_H8_TILE.dat` and places three rotated copies (copy 0/1/2) around C_{13} to form a single 3-fold symmetric “super cluster”. The whole assembly is rotated by 180 so that it emerges from `rotsym.c` the right way up.
2. **Level-match overlay.** For each ring level $L = 1\dots 5$, reads the symmetric completions produced by `grow_symmetric2` (`data/sym12/sym_completions_levelN.dat`) and finds the record whose interior best matches the super cluster. Uses the level-5 match to fix a global rigid-body transform (rotation + optional reflection) from grow-space into super-cluster space.
3. **SVG image.** Renders the super cluster (anti-hats in yellow, hats in pale blue) with a semi-transparent level-coloured overlay and a bold outer boundary, written to `from_python/h8_rot_overlay.svg`.
4. **CSV export.** Writes two files:
 - `from_python/SUPER_SUPER_H8_ROT.csv` — all three copies one after another.
 - `from_python/LEVEL_ROT.csv` — the grown cluster, ordered inside-out by level.

Background: Hat Tile Encoding

CSV Parameter Format

Each hat tile in the project is stored as five numbers (x, y, dir, ref, pt):

Field	Meaning
x, y	World-space anchor: the position in $\mathbb{R}^2$ of vertex pt of the tile.
dir	Discrete orientation in the range $0\dots 11$ (each step is 30; positive = CCW for regular hats, CW for reflected).
ref	Chirality flag: False = regular hat, True = anti-hat (reflected).
pt	Index $1\dots 14$ of the vertex whose position is given by (x, y) . Acts as the “pivot” for the local orientation frame.

The five parameters are sufficient to reconstruct the full 14-vertex polygon in world space; see `real_world` below.

Edge Lengths and Directions in `hv14`

The hat tile is a 14-vertex (13-edge) polygon on the tetrille lattice. Its edge lengths alternate between 1 and $\sqrt{3}$, and its turning angles at each vertex are multiples of 30.

Two lookup arrays encode the canonical hat boundary:

```
raw_len = [1,1,0,0,1,1,0,0,1,1,0,0,0,0] # 1=sqrt(3), 0=1
raw_rot = [-1,1,4,6,3,5,8,6,9,7,10,12,12,14]
```

`raw_len[j] == 1` means edge j has length $\sqrt{3}$; 0 means length 1. `raw_rot[j]` encodes the cumulative heading (in 30 units) at the end of edge j in the canonical, unreflected orientation.

Hat Geometry Functions

hv14 — Local 14-Vertex Polygon

hv14(d, ref, pt) — build the 14 vertices

Given a discrete direction d , a reflection flag ref , and a pivot vertex index pt , `hv14` computes the 14 vertices of the hat in a local coordinate frame with the pivot at the origin.

The loop over $j = 0 \dots 13$ reads each edge's properties from the two lookup arrays, applying a cyclic shift and optional reversal for reflected tiles. For a reflected tile the traversal order is reversed $((pt+13-j) \% 14)$ and all rotations are negated $((-rot) \% 12)$.

The heading α_j at step j is

$$\alpha_j = \pi (erot[j] + 1)/6$$

so that steps of 1 in `erot` correspond to 30 turns. Vertex positions are accumulated by integrating these headings.

The final reordering $(i-pt) \% 14$ shifts the vertex array so that index 0 in the output corresponds to the pivot vertex.

```
R3 = math.sqrt(3.0)

def hv14(d, ref, pt):
    elen = [0.0]*14; erot = [0]*14
    for j in range(14):
        src = (pt+(13-j))%14 if ref \
            else (pt+j)%14
        elen[j] = R3 if raw_len[src] else 1.0
        rot = raw_rot[src] + d
        erot[j] = (-rot)%12 if ref \
            else rot%12
    vx=[0.0]*15; vy=[0.0]*15
    for j in range(14):
        a = math.pi*(erot[j]+1)/6.0
        vx[j+1]=vx[j]+elen[j]*math.cos(a)
        vy[j+1]=vy[j]+elen[j]*math.sin(a)
    return np.array(
        [(vx[(i-pt)%14], vy[(i-pt)%14])
         for i in range(14)])
```

real_world — CSV Parameters to World Coordinates

real_world(d, ref, piv, x, y)

Converts the five CSV parameters into an array of 14 world-space vertices.

The trick is that `hv14` already places the pivot vertex at the origin of the local frame. `real_world` therefore:

1. Calls `hv14` with direction ± 1 (the “natural” direction for the reflected/non-reflected case) and pivot $piv = (pt-1)\%14$.
2. Builds the 2×2 rotation matrix R for angle $\theta = dir \cdot \pi/6$ (negated for reflected tiles).
3. Returns $V \cdot R^T + (x, y)$: rotate the local-frame vertices and translate to the anchor.

The result is an $[14 \times 2]$ array of (x, y) world positions.

```
def real_world(d, ref, piv, x, y):
    V = hv14(1 if ref else -1, ref, piv)
    th = d*(math.pi/6.0)*(-1 if ref else 1)
    c = math.cos(th); s = math.sin(th)
    Rm = np.array([[c, -s], [s, c]])
    return V@Rm.T + np.array([x, y])
```

cworld — Tetrille Integer Coordinates to World Space

The grow-overlay data is stored as tetrille integer triples (v, x, y) where $v \in \{3, 4, 6\}$ is the vertex valence. `cworld` converts to Euclidean $\mathbb{R}^2$:

```
def cworld(v, x, y):
    # tetrille Coord -> world
    if v == 6:    sx, sy = 6*x, 6*y
    elif v == 4: sx, sy = 3*x, 3*y
    else:        sx, sy = 2*(x-y), 2*(x+2*y)
    return ((3-sx-sy)/2.0, (sy-sx+3)/(2*R3))
```

Each valence system has a different basis in terms of the underlying $v = 6$ units: $v = 4$ points are at half-spacing, and $v = 3$ points lie on the triangular sublattice at a further $1/\sqrt{3}$ rescaling. The formula converts to a shared Euclidean frame where the canonical hat tile fits alongside the `hat.tile` SVG output.

IO Functions

load_csv — Reading Hat Placement Files

`load_csv(fn)` — load hat records from CSV

Reads a CSV file whose data rows are of the form `x,y,dir,ref,pt`. Lines are skipped if they do not begin with a digit, sign, or decimal point — this quietly ignores the header line and any comment lines.

Each accepted row is parsed into a Python dict with keys `x`, `y`, `dir`, `ref`, `pt`, and a pre-computed `world` array (14×2 float array) obtained by calling `real_world(d, ref, (pt-1)%14, x, y)`. The $(pt-1)\%14$ shift converts the 1-based CSV pivot index to the 0-based index expected by `hv14`.

```
def load_csv(fn):
    out = []
    for ln in open(fn):
        s = ln.strip()
        if not s or s[0] not in \
            "-+.0123456789":
            continue
        p = s.split(",")
        x,y = float(p[0]),float(p[1])
        d = int(p[2])
        ref = 1 if p[3][0] in "Tt1" else 0
        pt = int(p[4])
        out.append(dict(
            x=x, y=y, dir=d, ref=ref, pt=pt,
            world=real_world(
                d, ref, (pt-1)%14, x, y)))
    return out
```

load_records — Reading Grow-Symmetric2 Output

`load_records(level)` — parse .dat completion files

Reads `data/sym12/sym_completions_level{L}.dat`, which is the output of `grow_symmetric2`. Each record in that file contains a `tiles:` field whose value is a list of hat cycles, each cycle being a list of tetrille integer triples.

The function uses two nested regex passes:

1. `tiles:(\[ \[.*?\]\])` — extracts the bracketed tiles string for each record.
2. `\[((?:\(-?\d+,-?\d+,-?\d+\),?)++)\]` — splits into individual hat cycle strings.
3. `\((-?\d+),(-?\d+),(-?\d+)\)` — extracts each vertex triple (v, x, y) .

Each triple is converted to world space via `cworld`, yielding a list of $[14 \times 2]$ float arrays — one per hat tile per record.

```
def load_records(level):
    txt = open(
        f"data/sym12/"
        f"sym_completions_level{level}.dat"
    ).read()
    recs = []
    for line in re.findall(
        r'tiles:(\[ \[.*?\]\])', txt):
        hats = [
            np.array([cworld(*map(int,m))
                for m in re.findall(
                    r'\((-?\d+),(-?\d+),(-?\d+)\)',
                    hat)])
            for hat in re.findall(
                r'\[((?:\(-?\d+,-?\d+,-?\d+\),?)++)\]', line)]
        recs.append(hats)
    return recs
```

Super Cluster Construction

Rotation Constants

Three constants control the assembly of the super cluster:

```
C13          = np.array((-68.5, 0.8660254037844386)) # arrangement #13 centre
COPY_ANGLES = [180.0, 300.0, 60.0]               # 0/120/240 + 180 deg display rotation
```

C_{13} is the world-space position of the $v[12]$ -type triple vertex for arrangement #13 — the gap-free H8 centre. The three copy angles encode a 120-step rotation (0/120/240) combined with a global +180 display rotation that corrects for the orientation convention used by `rotsym.c`.

rotate_param — Rotating a CSV Hat Record

rotate_param(h, deg, ctr) — rotate placement parameters

Rotating a hat tile in *world space* by angle `deg` about a centre `ctr` must be reflected in the CSV parameters (x, y, dir) .

The `dir` field counts 30 steps, so the rotation step count is $u = deg/30$. The script asserts this is an integer. For a regular hat the direction increases CCW ($+u$); for a reflected hat the direction increases CW ($-u$), because the chirality flip reverses the angular convention.

The pivot position (x, y) is rotated geometrically: $(nx, ny) = ctr + R_{deg} \cdot ((x, y) - ctr)$.

A geometric cross-check (not in the production loop but run during verification) confirms that `r["world"]` matches the directly-rotated world array to within $\sim 10^{-13}$.

```
def rotate_param(h, deg, ctr):
    u = deg / 30.0
    assert abs(u-round(u)) < 1e-9
    u = int(round(u))
    nd = (h["dir"] +
          (-u if h["ref"] else u)) % 12
    nx, ny = ctr + rotmat(deg) @ (
        np.array([h["x"], h["y"]]) - ctr)
    return dict(
        x=float(nx), y=float(ny),
        dir=nd, ref=h["ref"], pt=h["pt"],
        world=real_world(
            nd, h["ref"],
            (h["pt"]-1)%14, nx, ny))
```

Assembling the Three Copies

The base tile set is loaded from `from_python/SUPER_SUPER_H8_TILE.dat`. Each of the three copy angles is applied to every hat in the base:

```
base = load_csv("from_python/SUPER_SUPER_H8_TILE.dat")
copies = []
for ci, ang in enumerate(COPY_ANGLES):
    copy = []
    for h in base:
        r = rotate_param(h, ang, C13)
        # geometric cross-check
        ref_world = (h["world"]-C13) @ rotmat(ang).T + C13
        maxerr = max(maxerr, np.abs(r["world"]-ref_world).max())
        r["copy"] = ci
        copy.append(r)
    copies.append(copy)
super_hats = [h for cp in copies for h in cp]
```

The cross-check verifies that the parameter-level rotation matches the directly-rotated world vertices to numerical precision. The flat list `super_hats` then contains all three copies concatenated; its length is $3 \times |base|$. The centroid array `Scent = np.array([h["world"].mean(0) for h in super_hats])` stores the 2D centre of mass of every hat's 14 vertices, used as the target point set for the alignment step.

Grow Overlay: Alignment Algorithm

Overview

The grow-symmetric2 completions live in a coordinate frame centred at

$$g_{ctr} = cworld(3, 1, -1)$$

which is the tetrille world-space position of $v[12]$ in the canonical hat tile. This is *not* the same as C_{13} ; a rigid-body transform (rotation and optional reflection) must be found to map grow-space centroids onto super-cluster centroids.

The function `fullalign` performs this alignment using a procedure analogous to the Iterative Closest Point (ICP) algorithm with a global initialisation sweep.

fullalign — ICP-Style Centroid Alignment

`fullalign(hats)` — find best rigid transform

Input: a list of $[14 \times 2]$ world arrays from one grown completion record. Centroids G are computed by averaging each hat's 14 vertices.

Outer loop (reflection) iterates over `refl` $\in \{+1, -1\}$. When `refl` = -1 the x -coordinates of the zero-centred point set P are negated before any rotation, implementing a mirror.

Middle loop (initial angle) sweeps the starting rotation in 3 steps from 0 to 360.

Inner loop (ICP refinement): up to 6 SVD-based iterations. At each iteration:

1. Rotate P by the current estimate R and add C_{13} to get candidate world positions T .
2. For each point in T find its nearest neighbour in `Scent` (squared distances).
3. Build the cross-covariance matrix $H = A^T B$ where A are the matched source points and B the corresponding target points.
4. Solve $H = U \Sigma V^T$ (SVD) and set $R_k = V^T U^T$ (orthogonal Procrustes).
5. Extract the updated angle from R_k .

The final coverage fraction f (share of grown points landing within distance $\sqrt{6 \times 10^{-2}}$ of a super-cluster centroid) is used to rank candidates. The function returns the best $(f, (refl, R))$.

```
gctr = np.array(cworld(3, 1, -1))

def fullalign(hats):
    G = np.array([h.mean(0) for h in hats])
    best = (-1, None)
    for refl in (1, -1):
        P = (G-gctr).copy()
        P[:, 0] *= refl
        for d0 in range(0, 360, 3):
            deg = float(d0)
            for _ in range(6):
                Rm = rotmat(deg)
                T = P@Rm.T + C13
                d2 = ((T[:,None] - Scent[None])**2
                     ).sum(-1)
                nn = d2.argmin(1)
                msk = d2.min(1) < 6e-2
                if msk.sum() < 3: break
                A = P[msk]
                B = Scent[nn[msk]] - C13
                H = A.T@B
                U, _, Vt = np.linalg.svd(H)
                Rk = Vt.T@U.T
                deg = math.degrees(
                    math.atan2(Rk[1,0], Rk[0,0]))
                Rm = rotmat(deg)
                T = P@Rm.T + C13
                f = (((T[:,None] - Scent[None])**2
                     ).sum(-1).min(1) < 6e-2).mean()
                if f > best[0]:
                    best = (f, (refl, Rm))
    return best
```

The threshold 6×10^{-2} is chosen to be smaller than the minimum inter-hat distance in world space, ensuring that each match is unambiguous.

Locking the Transform from Level-5

The global rigid-body transform is fixed once, using the level-5 completion record that achieves the best coverage:

```
best5 = (-1, None, None)
for i, rec in enumerate(load_records(5)):
    f, tr = fullalign(rec)
```

```

    if f > best5[0]: best5 = (f, i, tr)
f5, idx5, (refl, Rm) = best5

```

All subsequent per-level lookups use the same (refl, R_m) pair via the closure `place(hats)`, which applies the fixed reflection + rotation and translates to C_{13} :

```

def place(hats):
    out = []
    for h in hats:
        p = (h-gctr).copy(); p[:, 0] *= refl
        out.append(p@Rm.T + C13)
    return out

```

Per-Level Matching and Birth Assignment

Once the transform is fixed, the script iterates over levels $L = 1 \dots 5$. For each level it finds the completion record that covers the most of `Scent` and records which super-cluster hat first appears at that level:

```

birth = {}                                # super-hat index -> first level
for L in range(1, 6):
    bestrec = (-1, None, None)
    for i, rec in enumerate(load_records(L)):
        placed = place(rec)
        pc = np.array([h.mean(0) for h in placed])
        cov = ((pc[:,None]-Scent[None])**2).sum(-1).min(1) < 6e-2
        if cov.mean() > bestrec[0]:
            bestrec = (cov.mean(), i, placed)
    frac, ri, placed = bestrec
    pc = np.array([h.mean(0) for h in placed])
    nn = ((pc[:,None]-Scent[None])**2).sum(-1)
    for j in range(len(placed)):
        si = nn[j].argmin()
        if nn[j, si] < 6e-2 and si not in birth:
            birth[si] = L

```

After the loop, `birth[si]` gives the *birth level* of super-cluster hat `si`: the earliest ring at which that hat is covered by a symmetric grown completion. This assignment drives the level-coloured overlay in the SVG.

SVG Rendering

Coordinate Scaling

The script computes a uniform pixel scale `SC` such that the widest dimension of the super cluster spans exactly 1400 pixels, with a margin `M = 24` on each side:

```

allw = np.vstack([h["world"] for h in super_hats])
mnx, mny = allw.min(0); mxx, mxy = allw.max(0)
W, H = mxx-mnx, mxy-mny; span = max(W, H)
SC = 1400 / span
PW, PH = W*SC+2*M, H*SC+2*M
def px(p): return (p[0]-mnx)*SC+M
def py(p): return PH-((p[1]-mny)*SC+M) # SVG y flipped

```

The y -axis is flipped (`py`) because SVG coordinates increase downward while mathematical coordinates increase upward.

Drawing Layers

The SVG is built in four layers (painter's order, back to front):

1. **Super cluster tiles.** Every hat in `super_hats` is drawn as a closed 14-vertex polygon:
 - Anti-hats (`ref=1`): fill `#f4d03f` (golden yellow).
 - Regular hats (`ref=0`): fill `#c5d6ea` (pale steel blue).
 - Stroke: `#7a8aaa` at width 0.3 px.
2. **Level-coloured overlay.** For each super-hat index `si` with a known birth level L , the same polygon is redrawn with fill colour from a 5-step magma-ish palette and 50% opacity:

Level	Hex	Colour
1	<code>#3b0f70</code>	deep purple
2	<code>#641a80</code>	violet
3	<code>#a52c60</code>	magenta
4	<code>#de4968</code>	coral
5	<code>#fe9f6d</code>	peach

The 50% opacity lets the anti-hat yellow show through where anti-hats receive an overlay.

3. **Boundary.** The outer boundary of the grown cluster (`birth.keys()`) is computed by `boundary_edges` and drawn as bold black lines (width 2.2 px).
4. **Legend and centre marker.** A small colour legend for levels 1–5 and the anti-hat colour. A red circle of radius 6 px marks C_{13} .

`boundary_edges` — Outer Boundary of a Tile Subset

`boundary_edges(idxs)` — extract the boundary

A boundary edge of a planar tile union is an edge that is shared by exactly one tile. For each edge in the 14-vertex boundary of each selected tile, the function records it in a canonical form (sorted endpoint key pair) and counts how many tiles share it. Edges with count = 1 are boundary edges.

Vertex coordinates are rounded to 10^{-4} before hashing to avoid floating-point near-misses from independent computations of the same geometric point.

```
def boundary_edges(idxs):
    cnt = defaultdict(int); seg = {}
    def k(p):
        return (round(p[0]*1e4),
                round(p[1]*1e4))
    for si in idxs:
        h = super_hats[si]["world"]
        for i in range(14):
            a=h[i]; b=h[(i+1)%14]
            ka=k(a); kb=k(b)
            e=(ka,kb) if ka<=kb \
                else (kb,ka)
            cnt[e]+=1; seg[e]=(a,b)
    return [seg[e]
            for e,c in cnt.items()
            if c == 1]
```

CSV Export

File Format

Both output CSV files share the same header and row format:

```
x,y,dir,ref,pt
-68.500000000000,0.866025403784,7,False,1
...
```

Coordinates are written with 12 significant figures via `%g` formatting; `ref` is written as `True/False`.

SUPER_SUPER_H8_ROT.csv

Contains all hats from `super_hats` (three copies concatenated in the order copy 0, copy 1, copy 2). This is the input consumed by `rotsym.c` and downstream C programs that need the full 3-fold cluster in CSV form.

LEVEL_ROT.csv

Contains only the hats covered by the grow overlay (those in `birth`), sorted inside-out: first by birth level, then by Euclidean distance from C_{13} within each level:

```
order = sorted(birth.keys(),
               key=lambda si: (birth[si],
                              np.hypot(*(Scent[si]-C13))))
write_csv("from_python/LEVEL_ROT.csv",
         [super_hats[si] for si in order])
```

This ordering makes it natural to stream the file and animate ring-by-ring growth: reading the file from top to bottom adds tiles from the centre outward.

Architecture Summary

```
constants: C13=(-68.5, 0.866), COPY_ANGLES=[180,300,60]

1. SUPER CLUSTER
load_csv("from_python/SUPER_SUPER_H8_TILE.dat") -> base (N hats)
for ci, ang in enumerate(COPY_ANGLES):
    for h in base: rotate_param(h, ang, C13) -> copies[ci]
super_hats = copies[0] + copies[1] + copies[2] (3N hats)
Scent = centroid of each hat's 14 vertices (3N x 2)

2. GROW OVERLAY
gctr = cworld(3, 1, -1) (grow-space origin)
fullalign(rec) for each rec in load_records(5):
    ICP: refl in {+1,-1}, sweep 0..360 deg (3 deg steps),
        6 SVD-Procrustes iterations each
    -> best (fraction, (refl, Rm))
lock transform: place(hats) = (hats-gctr)*refl @ Rm.T + C13

for L in 1..5:
    for rec in load_records(L):
        placed = place(rec); cov = nearest-in-Scent < 6e-2
    best rec at L -> bestrec
    assign birth[si] = L for newly matched super-hat indices

3. SVG
boundary_edges(birth.keys()) -> outer boundary segments
draw super_hats: ref=1 -> #f4d03f, ref=0 -> #c5d6ea
draw overlay: birth[si]=L -> LV[L-1] at opacity 0.5
draw boundary: black, stroke-width=2.2
write from_python/h8_rot_overlay.svg

4. CSV EXPORT
write_csv(SUPER_SUPER_H8_ROT.csv, super_hats)
order = sorted by (birth[si], |Scent[si]-C13|)
write_csv(LEVEL_ROT.csv, [super_hats[si] for si in order])
```


Results

Running `python3 MyHatTileStuff/h8_rotcluster.py` from the project root prints:

```
building #13 super cluster (centre -68.5000,0.8660, +180 deg)...
78 hats (6 anti-hats); CSV-param vs geometry max error = 2.22e-16
grow alignment locked from level5 rec1 = 100.0%
level 1: rec 1 matches interior at 15.4% (cumulative covered hats=12)
level 2: rec 1 matches interior at 30.8% (cumulative covered hats=24)
level 3: rec 1 matches interior at 57.7% (cumulative covered hats=45)
level 4: rec 1 matches interior at 76.9% (cumulative covered hats=60)
level 5: rec 1 matches interior at 100.0% (cumulative covered hats=78)
wrote from_python/h8_rot_overlay.svg
wrote from_python/SUPER_SUPER_H8_ROT.csv (78 hats: 26+26+26)
wrote from_python/LEVEL_ROT.csv (78 hats; per-level {1: 12, 2: 12, 3: 21, 4: 15, 5: 18})
```

Key observations:

- The geometric cross-check error of 2.22×10^{-16} (machine epsilon) confirms that the parameter-level rotation in `rotate_param` is exactly consistent with the direct geometric rotation.
- The level-5 alignment achieves 100% coverage, confirming that the `grow-symmetric2` output at level 5 contains a completion that fills the entire H8 super cluster interior.
- The 78 hats split into 26 per copy (three-fold symmetry) and comprise 6 anti-hats total (2 per copy), consistent with the expected 3 : 1 hat-to-anti-hat ratio.

Usage

```
# Run from the project root (no arguments needed)
python3 MyHatTileStuff/h8_rotcluster.py

# Prerequisites
# from_python/SUPER_SUPER_H8_TILE.dat -- base H8 super tile (CSV)
# data/sym12/sym_completions_level{1-5}.dat -- grow-symmetric2 output

# Outputs
# from_python/h8_rot_overlay.svg -- rendered image
# from_python/SUPER_SUPER_H8_ROT.csv
# from_python/LEVEL_ROT.csv
```

The script requires only the Python standard library and NumPy. No SciPy or other geometry libraries are needed.

MyHatTileStuff/h8_rotcluster.py — part of PolyformTown. Compile this document with `pdflatex -shell-escape h8_rotcluster.tex` (requires `pygments` for minted).